

DP Concepts

video
39

&

Questions



codestorywithMIK

Do not be afraid of a slow start; every expert was once a beginner.

Consistency is the bridge between where you are and where you want to be.

Just keep showing up, and the results will take care of themselves.



MIK...



codestorywithmik



CSwithMIK



codestorywithMIK

- ✓ 1-D based DP
- ✓ Grid based DP
- ✓ String based DP
- ✓ Knapsack Series
 - Digit DP
 - Game Strategy

we'll do:-

(i) RECURSION
+
MEMOIZATION
(Top Down)

(ii) Bottom UP .

(iii) Time & Space

Knapsack Series

(video - 4)



Variants

- ↳ Fractional Knapsack ✓
- ↳ 0/1 Knapsack ✓
- ↳ Unbounded Knapsack

0-1

Knapsack

 DP CONCEPTS & QNS KNAPSACK SERIES VIDEO-1 WHY KNAPSACK IS A BIG DEAL IN DP ??? VIDEO-36 C++ JAVA 10:42	Introduction Knapsack Series Introduction Video 1 DP Concepts & Qns-36 codestorywithMIK ✓
 DP CONCEPTS & QNS KNAPSACK SERIES VIDEO-2 FRACTIONAL KNAPSACK VIDEO-37 C++ JAVA 21:41	Fractional Knapsack Knapsack Series Video 2 DP Concepts & Qns-37 codestorywithMIK ✓
 DP CONCEPTS & QNS KNAPSACK SERIES VIDEO-3 0-1 KNAPSACK PART-1 VIDEO-38 C++ JAVA 23:33 WHAT NO BODY TELLS YOU	0/1 Knapsack Recursion Two Ways To Code Knapsack Series DP Concepts & Qns-38 MIK ✓

Two ways to write Recursion...

```
int n;
int t[1001][1001];
int solve(int i, int W, vector<int>& val, vector<int>& wt) {
    if(i >= n || W == 0)
        return 0;

    if(t[i][W] != -1) {
        return t[i][W];
    }
    int take = 0;
    int skip = 0;

    //take
    if(wt[i] <= W) {
        take = val[i] + solve(i+1, W - wt[i], val, wt);
    }

    skip = solve(i+1, W, val, wt);

    return t[i][W] = max(take, skip);
}
int knapsack(int W, vector<int> &val, vector<int> &wt) {
    n = val.size();
    memset(t, -1, sizeof(t));
    return solve(0, W, val, wt);
}
```

Solve(i=0, W)

```
class Solution {
public:
    int t[1001][1001];
    int solve(int n, int W, vector<int>& val, vector<int>& wt) {
        if(n <= 0 || W == 0)
            return 0;

        if(t[n][W] != -1) {
            return t[n][W];
        }
        int take = 0;
        int skip = 0;

        //take
        if(wt[n-1] <= W) {
            take = val[n-1] + solve(n-1, W - wt[n-1], val, wt);
        }

        skip = solve(n-1, W, val, wt);

        return t[n][W] = max(take, skip);
    }
    int knapsack(int W, vector<int> &val, vector<int> &wt) {
        int n = val.size();
        memset(t, -1, sizeof(t));
        return solve(n, W, val, wt);
    }
};
```

Solve(n, W)

Bottom up

Bottom-up is just recursive code, but we replace function calls with table lookups.

State Definition

```

class Solution {
public:
    int t[1001][1001];
    int solve(int n, int W, vector<int>& val, vector<int>& wt) {
        if(n <= 0 || W == 0)
            return 0;

        if(t[n][W] != -1) {
            return t[n][W];
        }
        int take = 0;
        int skip = 0;

        //take
        if(wt[n-1] <= W) {
            take = val[n-1] + solve(n-1, W - wt[n-1], val, wt);
        }

        skip = solve(n-1, W, val, wt);

        return t[n][W] = max(take, skip);
    }
    int knapsack(int W, vector<int> &val, vector<int> &wt) {
        int n = val.size();
        memset(t, -1, sizeof(t));
        return solve(n, W, val, wt);
    }
};

```

$t[n+1][w+1]$
 $t[i][j] = \text{max profit when}$
 $\uparrow \quad \uparrow$ I have i items
and knapsack wt = j

n items, w knp.

return $t[n][W]$

Easiest way to derive Bottom-up
from Recursive Code

```

class Solution {
public:
    int t[1001][1001];
    int solve(int n, int W, vector<int>& val, vector<int>& wt) {
        if(n <= 0 || W == 0)
            return 0;

        if(t[n][W] != -1) {
            return t[n][W];
        }
        int take = 0;
        int skip = 0;

        //take
        if(wt[n-1] <= W) {
            take = val[n-1] + solve(n-1, W - wt[n-1], val, wt);
        }

        skip = solve(n-1, W, val, wt);

        return t[n][W] = max(take, skip);
    }
    int knapsack(int W, vector<int> &val, vector<int> &wt) {
        int n = val.size();
        memset(t, -1, sizeof(t));
        return solve(n, W, val, wt);
    }
};

```

$n == 0$ || $W == 0$

max profit = 0

$t[0][j] = 0$

$t[i][0] = 0$



i

0			
0			
6			

```

for (int j = 0; j < w+1; j++)
    t[0][j] = 0;

for (int i = 0; i < n+1; i++) {
    t[i][0] = 0;
}

```

Base Case
↓
Initialization

The final goal is to fill the dp table $t[][]$

```

for (i = 1; i < n+1; i++) { ← i items
    for (j = 1; j < w+1; j++) { ← j wt of prep.

```

```

        int take = 0;
        int skip = 0;

```

```

        if (w + [i-1] <= j) {

```

```

            take = val[i-1] + t[i-1][j - w + [i-1]];

```

```

        }

```

```

        skip = t[i-1][j];

```

T.C = $O(n \cdot w)$
S.C = $O(n \cdot w)$

```

    }
    f[i][j] = max(take, skip);
}

```

3

return f[n][w];

But we must also know
what's happening
behind the scene...

$n = 3$ items

val = { 1, 2, 3 }

wt = { 1, 2, 3 }

$W = 4$

$W=0$



weight of knap

0 1 2 3 4

$f[n+1][w+1]$

$\{v_1, v_2\}$
 $\{w_1, w_2\}$
 $w = 2$

0	0	0	0	0	0	...
→ 1	0	?	?			
2	0		?			
3	0					

$f[2][2]$

 $(n+1) \times (w+1)$

$i=2$ $j=2$

$$take = val[i-1] + f[i-1][j-w+1[i-1]]$$

$$val[i] + f[i][j-w_2]$$

$v_1, v_2 \rightarrow$

$$v_2 + \frac{f[i][j-w_2]}{\text{loop up}}$$

$f[i][j] =$ previous cells 2+ solve
or 2+ 2+ 2+ ←

